Midterm Project Sharing

Customized ISA Processor

iclab042 TZU-CHEN, YANG

Observation

Performance: Total Cycle x Clock Period x Area^2

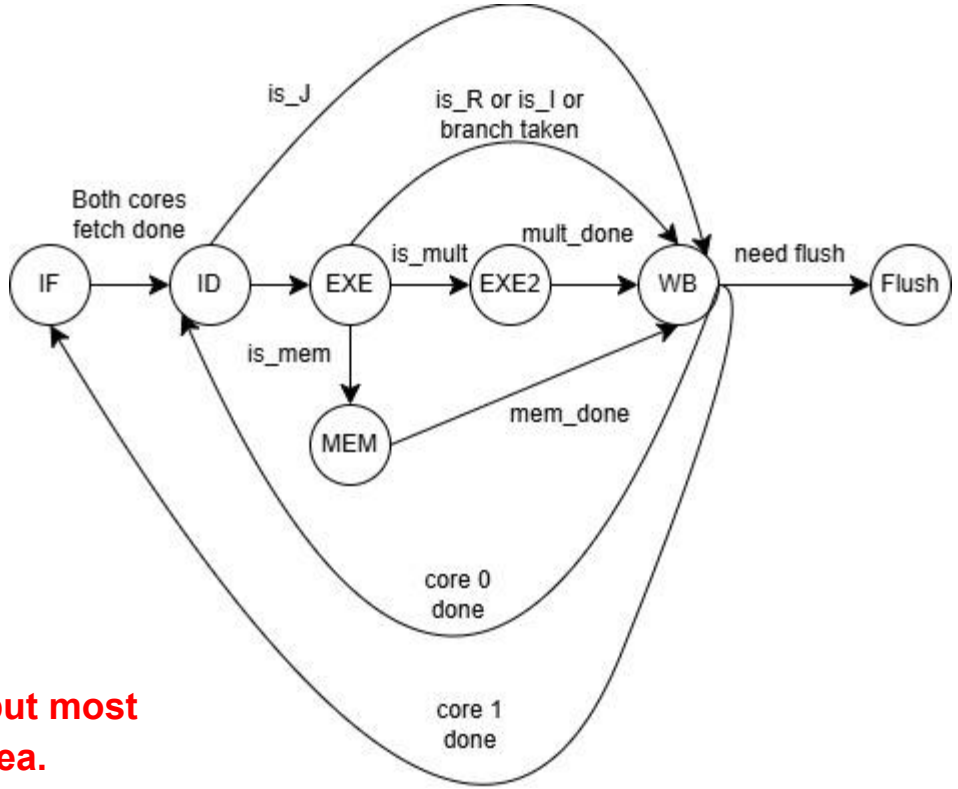
- Area is the dominant factor in the performance metric
- DRAM latency is very high
  - Latency is not that important
  - Lower cycle time is really benefitial

Overview

- Multi-cycle CPU
- Merge I-Cache and D-Cache
- Single execution and memory unit shared across both cores
- Shift-add multiplier
- Write-back cache with periodic flush
- My performance
  - Cycle time: 3.5 ns
  - Area: 203112.9
  - Latency: 56750

FSM

- One FSM drives both cores sequentially
- Execution order: core 0 -> core 1
    - Exception: when core 0 is Load and core 1 is Store, execution order is swapped
- stall_0 / stall_1 deasserted when state == WB and both cores have completed
- Flush triggered when instruction counter reaches 49 or PC hits 0x1FFE



There is still potential for latency optimization, but most improvements would increase cycle time and area.

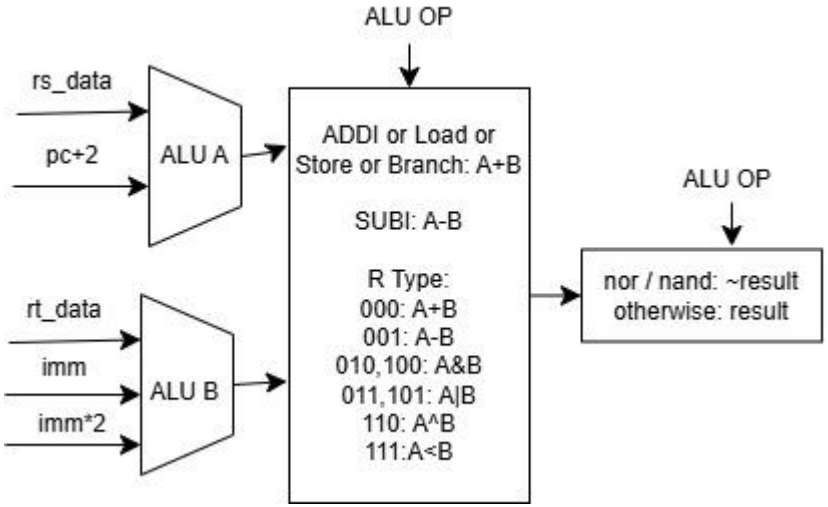
I-Cache + D-Cache

- Cache hierarchy
  - Address 0–127 : Core 0 instruction cache
  - Address 128–255 : Core 1 instruction cache
  - Address 256–511 : Shared data cache
- 1-entry instruction cache
- 2-entry direct-map write-back data cache
- Write-back condition: instruction counter reaches 49 or PC hits 0x1FFE

Combinations	Area
two 128X16 SRAM	~11000
two 256X16 SRAM	~13600
one 512X16 SRAM	96105.96
one 256X16 SRAM	68199.15

ALU

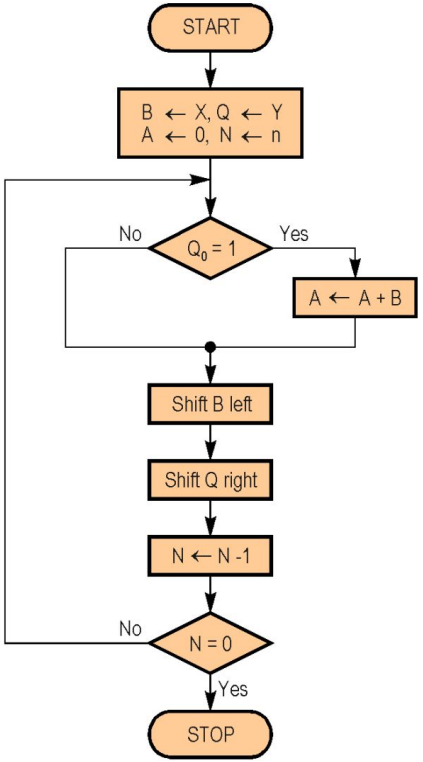
- All instructions share the same ALU
  - no extra adder (including branch target)
- NOR and NAND are implemented by inverting the AND/OR result



Shift-add multiplier

- 16-cycle shift-add multiplier
- Area cost ~25,000 vs ~50,000 for a standard multiplier — 50% reduction

Booth algorithm can reduce multiplier latency, but increases area in my design.



https://users.utcluj.ro/~baruch/book_ssce/SSCE-Shift-Mult.pdf

Cycle time optimization

- SRAM output latency is 3.5 ns, so cycle time can’t be lower than 3.5 ns
- Using shift-add or 3-stage pipelined multiplier can reduce multiplier critical path to be lower than 3.5 ns

```
Startpoint: core_1_memory_dcache_u_SRAM_512_16
            (rising edge-triggered flip-flop clocked by clk)
Endpoint: core_1_memory_data_o_reg_0_
          (rising edge-triggered flip-flop clocked by clk)
Path Group: clk
Path Type: max
```

Pattern

- Random test: Random instructions with forward branch/jump only
- Direct test: Edge case and for loop

Pseudo code:

```
core0:
int16_t &arr = 0x1100;
for i in range(0, 25):
    arr[i + 100] = arr[i] + arr[i + 25];

core1:
int16_t &arr = 0x1100;
for i in range(0, 25):
    arr[i + 125] = arr[i + 50] + arr[i + 75];
```

Assembly:

```
Register setup:
    core0: r1 = 128, r2 = 0, r3 = 25
    core1: r1 = 228, r2 = 0, r3 = 25

LOOP_TOP:
    BEQ  r2, r3, LOOP_END
    LOAD r4, 0(r1)
    LOAD r5, 25(r1)

    ADD  r6, r4, r5

    core0: NOP  / core1: STORE r6, 50(r1)
    core0: STORE r6, 50(r1) / core1: NOP

    ADDI r1, r1, 1
    ADDI r2, r2, 1
    JUMP LOOP_TOP

LOOP_END:
```

Evaluation

- Evaluated both 512×16 and 256×16 cache configurations
  - 256×16 showed significantly higher CPI on both random and directed tests — the area savings did not justify the performance loss

512×16 was selected as the final configuration

TABLE I: CPI comparison between different cache sizes

Test Type	512×16	256×16
Direct Test	57.06	157.65
Random Test	38.49	68.25

TABLE II: Area comparison between different cache sizes

Area Type	512×16	256×16
Combinational area	63520.94	60181.23
Buf/Inv area	12833.25	12420.78
Noncombinational area	43486.03	42042.37
Macro/Black Box area	96105.96	68199.15
Total cell area	203112.92	170422.75

Thank You